# TECNOLOGÍAS DE LA INFORMACIÓN
## Desarrollo de Sistemas Informáticos

**Profesor:**

Ing. **Ing. Jorge Arun Zambrano Cedeño, Mg.**

| Titulaciones | Semestre |
|---|---|
| • *Tecnologías de la Información en Línea* | *Quinto* |

**NOMBRES Y APELLIDOS: Rodolfo Ignacio Giler Ocaña**

**PARALELO: A**

**SEMESTRE: Quinto**

**ACTIVIDAD: #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal**

**PERIODO SEPTIEMBRE 2025 - ENERO 2026**

## 1. Introducción

Las organizaciones de protección animal gestionan un proceso con varias etapas y varios responsables: el reporte de un animal en situación de riesgo, su rescate, la atención veterinaria, la publicación para adopción, la evaluación de los interesados y el seguimiento posterior. Cuando este proceso se lleva en hojas de cálculo o mensajes dispersos, se pierde trazabilidad, se duplican registros y resulta difícil saber en qué estado se encuentra cada animal.

Este documento presenta el diseño de la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal que centraliza el ciclo de vida completo del animal. Incluye el análisis de actores y requerimientos, el modelado UML (casos de uso y clases), la aplicación de cuatro patrones de diseño y el registro de avances mediante el flujo de trabajo Gitflow en GitHub. El objetivo es diseñar la arquitectura de un sistema escalable y mantenible aplicando patrones de diseño y buenas prácticas de ingeniería de software.

## 2. Análisis del escenario

### 2.1 Descripción del escenario

Un refugio recibe reportes de animales abandonados o en peligro. Un rescatista acude, registra al animal y lo ingresa al refugio. Un veterinario lo evalúa y registra su tratamiento. Cuando el animal está en condiciones, el administrador lo publica para adopción. Los interesados consultan el catálogo y envían solicitudes que el administrador evalúa; si se aprueban, se formaliza la adopción. Después se realiza un seguimiento para verificar el bienestar del animal. En cada cambio importante se notifica a las personas involucradas.

El ciclo de vida de un animal en el sistema pasa por los estados: rescatado → en tratamiento → disponible → en proceso de adopción → adoptado.

## 2.2 Actores

| Actor | Tipo | Responsabilidad principal |
|---|---|---|
| Rescatista | Primario | Registrar reportes de rescate y animales rescatados |
| Veterinario | Primario | Registrar evaluaciones médicas y actualizar el estado del animal |
| Administrador | Primario | Publicar animales, evaluar y formalizar adopciones, gestionar usuarios y reportes |
| Adoptante | Primario | Consultar animales, solicitar adopción y recibir seguimiento |
| Voluntario | Secundario | Registrar turnos de cuidado |

## 2.3 Requerimientos funcionales

| Código | Requerimiento |
|---|---|
| RF01 | Registrar reportes de rescate con ubicación, fecha y descripción. |
| RF02 | Registrar animales rescatados con sus datos básicos (especie, nombre, edad, sexo, fecha de ingreso). |
| RF03 | Registrar fichas médicas (diagnóstico y tratamiento) asociadas a un animal. |
| RF04 | Controlar el estado del animal: rescatado, en tratamiento, disponible, en proceso de adopción y adoptado. |
| RF05 | Publicar animales disponibles y consultarlos mediante un catálogo. |
| RF06 | Permitir a un adoptante registrar una solicitud de adopción. |
| RF07 | Evaluar solicitudes con distintos criterios de evaluación. |
| RF08 | Formalizar la adopción y generar el contrato correspondiente. |
| RF09 | Registrar el seguimiento post-adopción. |
| RF10 | Notificar a los interesados los cambios de estado relevantes. |
| RF11 | Gestionar usuarios, autenticación y roles. |
| RF12 | Generar reportes (animales por estado, adopciones por período, rescates por zona). |
| RF13 | Registrar turnos de cuidado de voluntarios. |

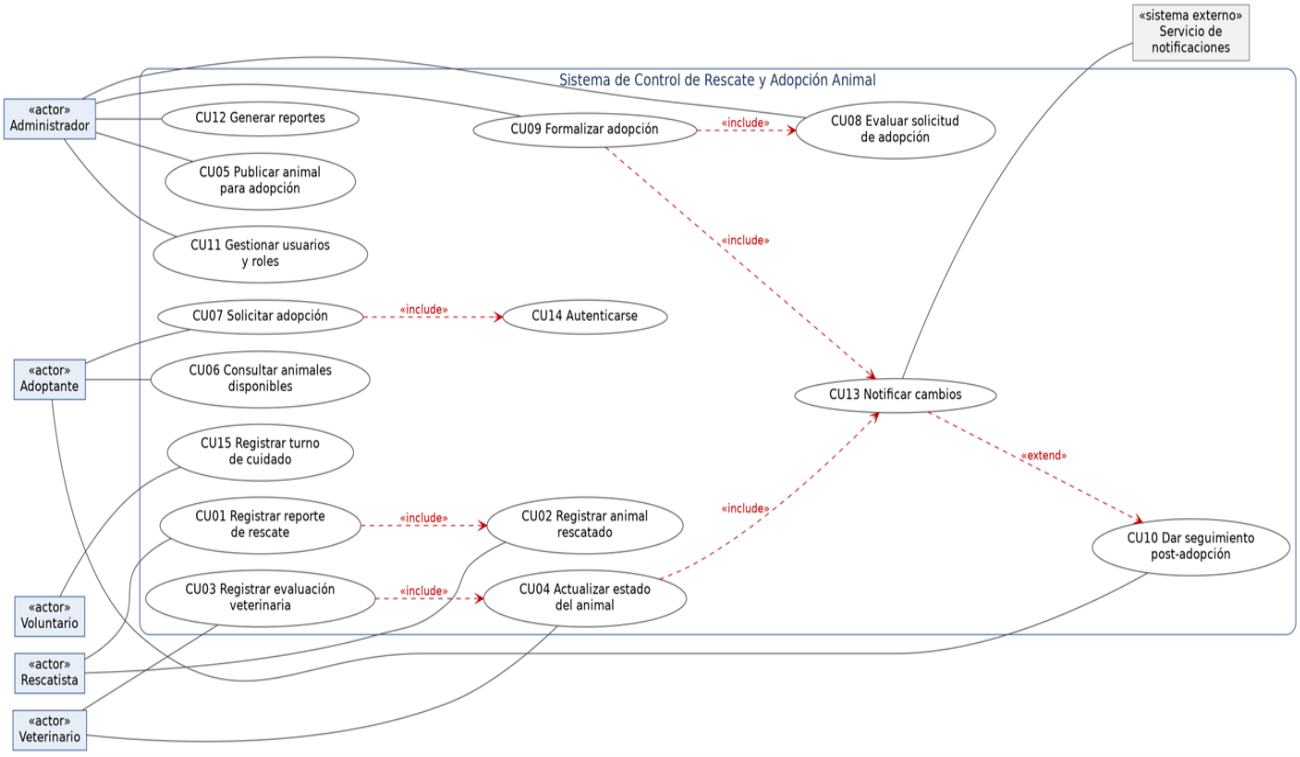
## 2.4 Requerimientos no funcionales

| Código | Categoría | Requerimiento |
|---|---|---|
| RNF01 | Seguridad | Acceso mediante autenticación y control de permisos por rol. |
| RNF02 | Trazabilidad | Todo cambio de estado de un animal debe quedar registrado con fecha y responsable. |
| RNF03 | Escalabilidad | Debe ser posible agregar nuevas especies o criterios |

| | | |
|---|---|---|
| | | de evaluación sin modificar el código existente. |
| RNF04 | Mantenibilidad | El diseño debe ser modular, con bajo acoplamiento y alta cohesión. |
| RNF05 | Disponibilidad | La consulta pública del catálogo debe estar disponible de forma continua. |
| RNF06 | Usabilidad | La interfaz debe permitir completar un registro de rescate en pocos pasos. |
| RNF07 | Interoperabilidad | La integración con servicios externos de mensajería debe estar aislada del núcleo del sistema. |

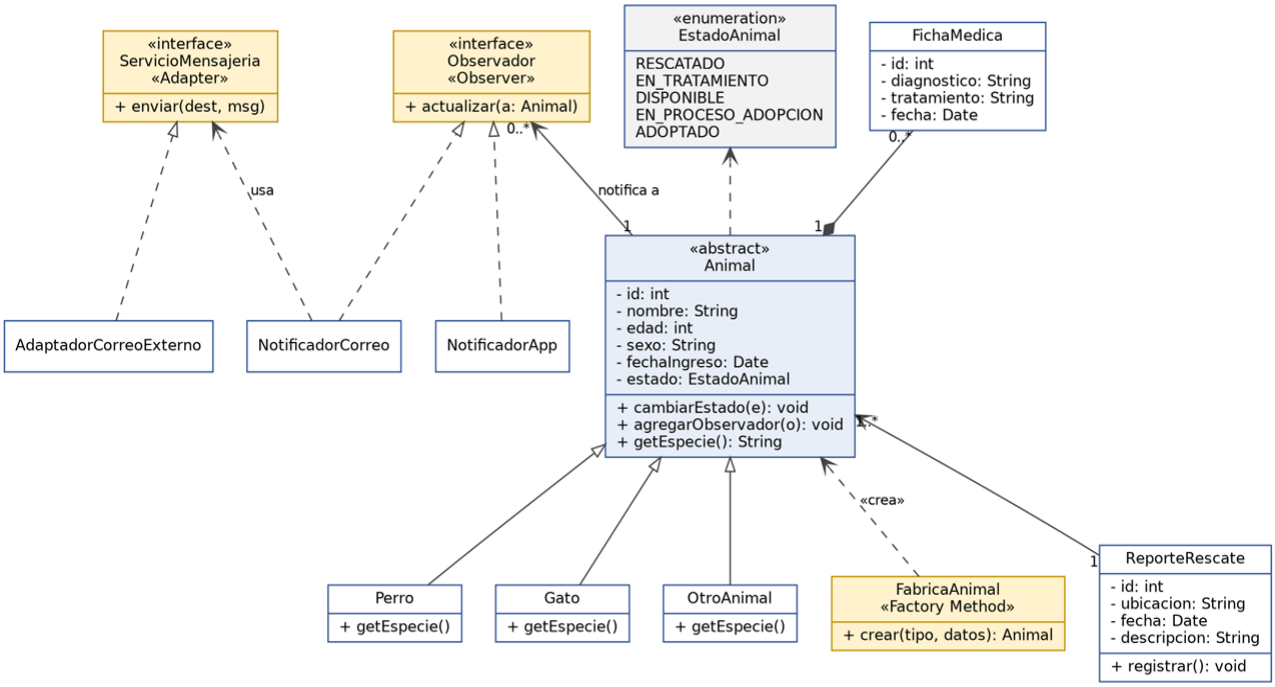
## 3. Modelado UML

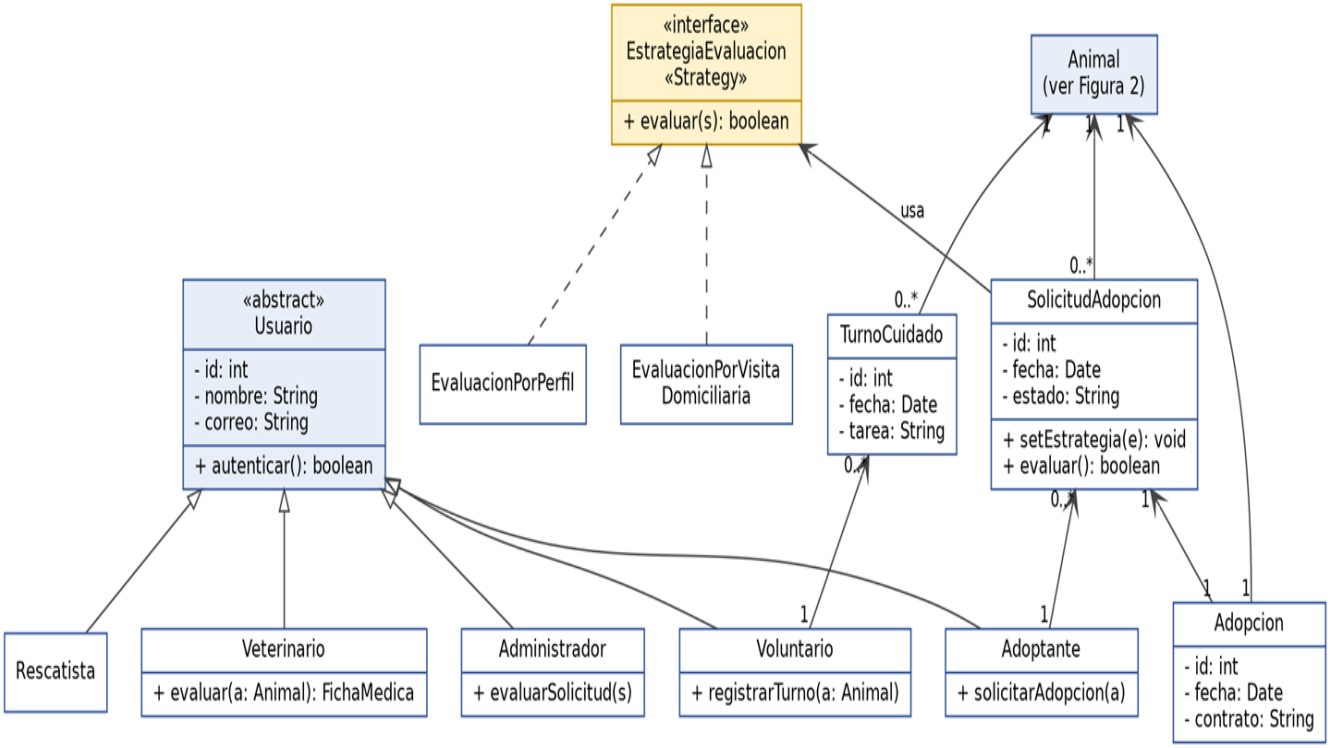
El diagrama contiene seis actores y quince casos de uso dentro del límite del sistema. Se utilizan relaciones include para comportamientos obligatorios (por ejemplo, registrar un reporte incluye registrar el animal, y formalizar una adopción incluye evaluar la solicitud) y extend para comportamientos opcionales.



## 3.2 Diagrama de clases

El diagrama de clases se divide en dos figuras para mantenerlo legible. La Figura 2 muestra el núcleo del dominio (animales, fichas médicas, estados) junto con las clases que implementan los patrones Factory Method, Observer y Adapter. La Figura 3 muestra los usuarios del sistema, el proceso de adopción y el patrón Strategy. Las clases que resuelven un patrón de diseño aparecen resaltadas en amarillo.



## 4. Patrones de diseño aplicados

Se aplican cuatro patrones, pertenecientes a las tres categorías (creacional, de comportamiento y estructural), para resolver necesidades concretas del sistema. Los ejemplos de código están en Java, aunque los patrones son independientes del lenguaje.

### 4.1 Factory Method (creacional)

Problema: el sistema registra animales de distintas especies y a futuro podrían agregarse más. Si el código cliente crea cada clase directamente, queda acoplado a todas las clases concretas.

Solución: FabricaAnimal centraliza la creación y devuelve un Animal según el tipo solicitado.

Beneficios: bajo acoplamiento y cumplimiento del principio abierto/cerrado (RNF03).

```
public class FabricaAnimal {
    public static Animal crear(String tipo, Map<String, Object> datos) {
        switch (tipo.toLowerCase()) {
            case "perro": return new Perro(datos);
            case "gato":  return new Gato(datos);
            default:      return new OtroAnimal(datos);
        }
    }
}
```

### 4.2 Strategy (comportamiento)

Problema: el refugio puede evaluar solicitudes de adopción con criterios distintos (revisión del perfil, visita domiciliaria, entrevista) y esos criterios pueden cambiar.

Solución: la interfaz EstrategiaEvaluacion encapsula cada algoritmo, y SolicitudAdopcion recibe la estrategia que debe usar.

Beneficios: se evita un bloque condicional extenso y se pueden agregar criterios sin modificar SolicitudAdopcion (RF07, RNF03).

```
public interface EstrategiaEvaluacion {
    boolean evaluar(SolicitudAdopcion solicitud);
}

public class SolicitudAdopcion {
    private EstrategiaEvaluacion estrategia;
    public void setEstrategia(EstrategiaEvaluacion e) { this.estrategia = e; }
    public boolean evaluar() { return estrategia.evaluar(this); }
}
```

### 4.3 Observer (comportamiento)

Problema: cuando cambia el estado de un animal, varias partes deben enterarse (adoptante interesado, administrador, voluntarios) sin que Animal conozca los detalles de cada una.

Solución: Animal mantiene una lista de Observador y los notifica en cada cambio de estado.

Beneficios: desacopla el modelo de las notificaciones y facilita agregar nuevos canales (RF04, RF10, RNF02).

```
public abstract class Animal {
    private List<Observador> observadores = new ArrayList<>();
    private EstadoAnimal estado;

    public void agregarObservador(Observador o) { observadores.add(o); }

    public void cambiarEstado(EstadoAnimal nuevo) {
        this.estado = nuevo;
        for (Observador o : observadores) o.actualizar(this);
    }
}
```

### 4.4 Adapter (estructural)

Problema: el servicio externo de mensajería tiene su propia API, distinta de la interfaz que el sistema espera. Usarla directamente ataría el núcleo a un proveedor concreto.

Solución: AdaptadorCorreoExterno implementa ServicioMensajeria y traduce las llamadas hacia la API del proveedor.

Beneficios: si se cambia de proveedor, solo se reemplaza el adaptador (RNF07).

```
public interface ServicioMensajeria {
    void enviar(String destino, String mensaje);
}

public class AdaptadorCorreoExterno implements ServicioMensajeria {
    private ClienteCorreoExterno cliente = new ClienteCorreoExterno();
    public void enviar(String destino, String mensaje) {
        cliente.sendMail(destino, "Refugio", mensaje);
    }
}
```

El informe se registra en el repositorio https://github.com/Rgiler98/DeberDSI, creado en GitHub como base del proyecto. El flujo de trabajo Gitflow (ramas develop, feature/... y release/...) se aplicará en las siguientes etapas del desarrollo.

## 5. Conclusiones

El análisis de actores y requerimientos permitió delimitar el alcance del sistema y definir las responsabilidades de cada rol.

Los diagramas UML dan una visión común del comportamiento (casos de uso) y de la estructura (clases) antes de programar.

Los cuatro patrones aplicados (Factory Method, Strategy, Observer y Adapter) reducen el acoplamiento y dejan el sistema preparado para crecer, por ejemplo con nuevas especies, criterios de evaluación o canales de notificación.

El uso de un repositorio en GitHub permite conservar el informe y su historial de cambios, y deja establecida la base para aplicar el flujo de trabajo Gitflow en las siguientes etapas del proyecto

## 6. Referencias

Atlassian. (s. f.). Flujo de trabajo de Gitflow. Atlassian Git Tutorial.

https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

Driessen, V. (2010). A successful Git branching model [Entrada de blog]. nvie.com.

https://nvie.com/posts/a-successful-git-branching-model/

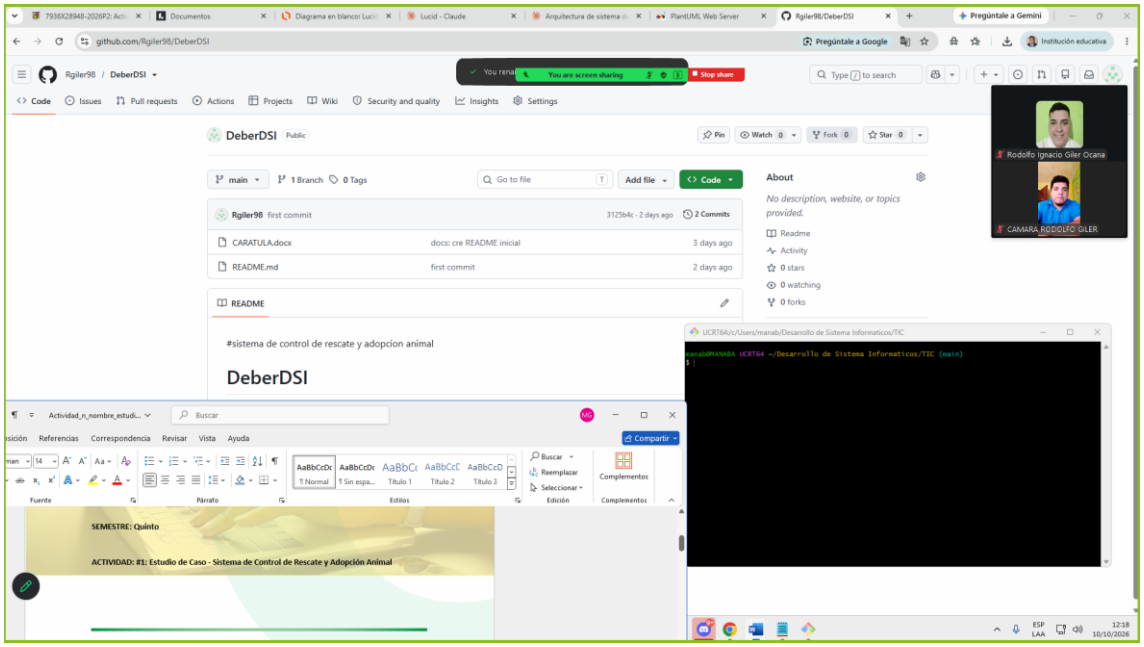
Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design patterns: Elements of

reusable object-oriented software. Addison-Wesley.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML) (Versión

2.5.1). https://www.omg.org/spec/UML/2.5.1/

# 7.EVIDENCIA




# 7.LINK DE REPOSITORIO GITHUB

https://github.com/Rgiler98/DeberDSI